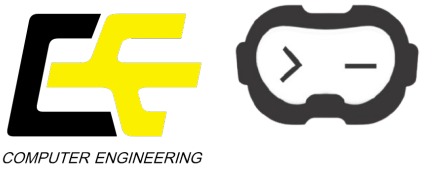


Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

Atika Najwa Azzahra - 5024241091

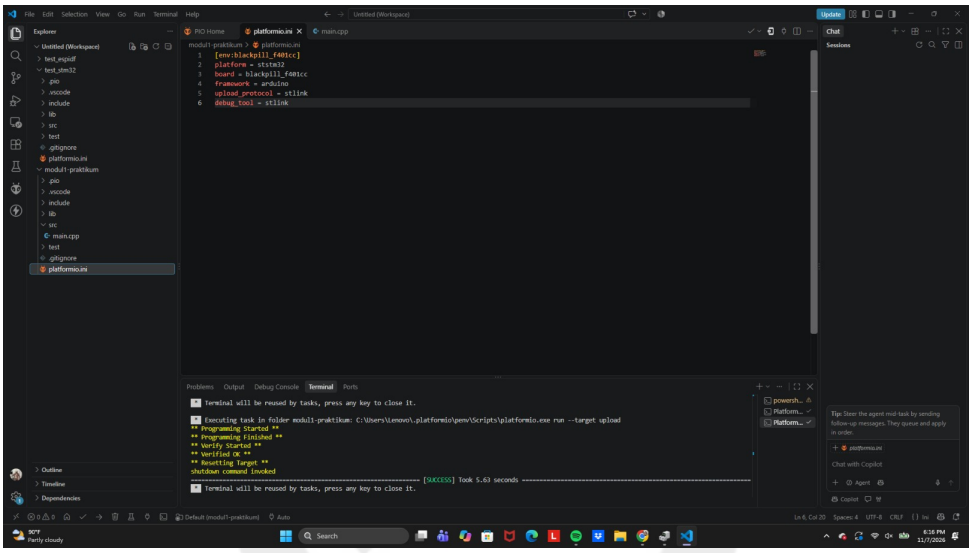
2026

1 Langkah-Langkah Percobaan

Praktikum Modul 1 terdiri dari lima percobaan. Seluruh percobaan dikerjakan dalam satu project PlatformIO yang sama, sehingga setiap pergantian percobaan cukup dilakukan dengan mengganti isi `platformio.ini` dan kode program pada folder `src`.

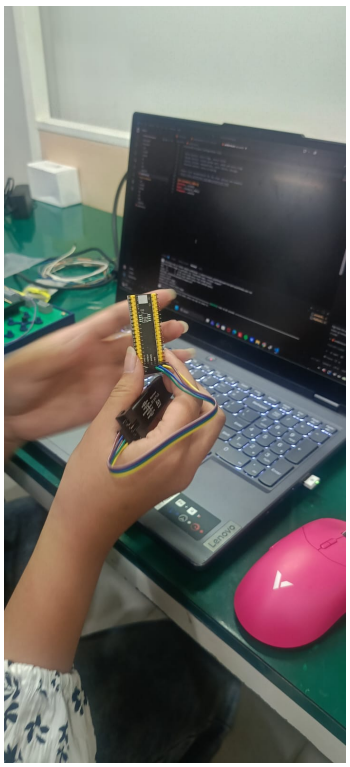
1.1 Percobaan 1: Blink STM32 Blackpill

1. Mengisi `platformio.ini` dengan board `blackpill_f401cc`, framework Arduino, serta `upload_protocol` dan `debug_tool` berupa `stlink`.



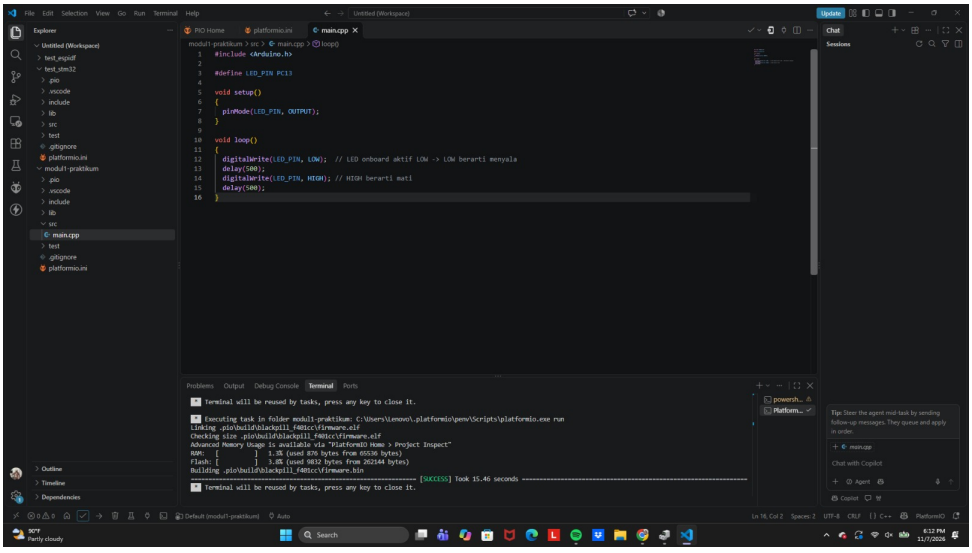
Gambar 1: Konfigurasi `platformio.ini` untuk STM32 Blackpill

2. Menghubungkan ST-Link ke Blackpill menggunakan empat kabel jumper female-female, yaitu SWDIO ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke 3V3. Setelah itu, ST-Link dihubungkan ke laptop melalui USB.



Gambar 2: Sambungan ST-Link ke STM32 Blackpill

3. Menulis kode Blink pada `src/main.cpp`, lalu melakukan Build dan Upload hingga terminal menampilkan status SUCCESS.



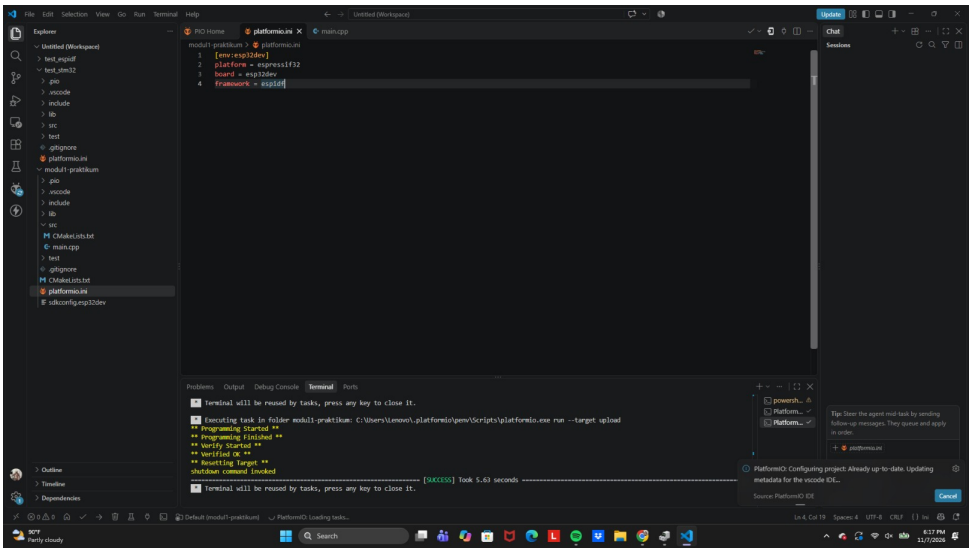
Gambar 3: Build dan Upload program Blink ke STM32 berhasil

4. Mengamati LED onboard pada pin PC13.

Gambar 4: LED onboard PC13 pada Blackpill berkedip

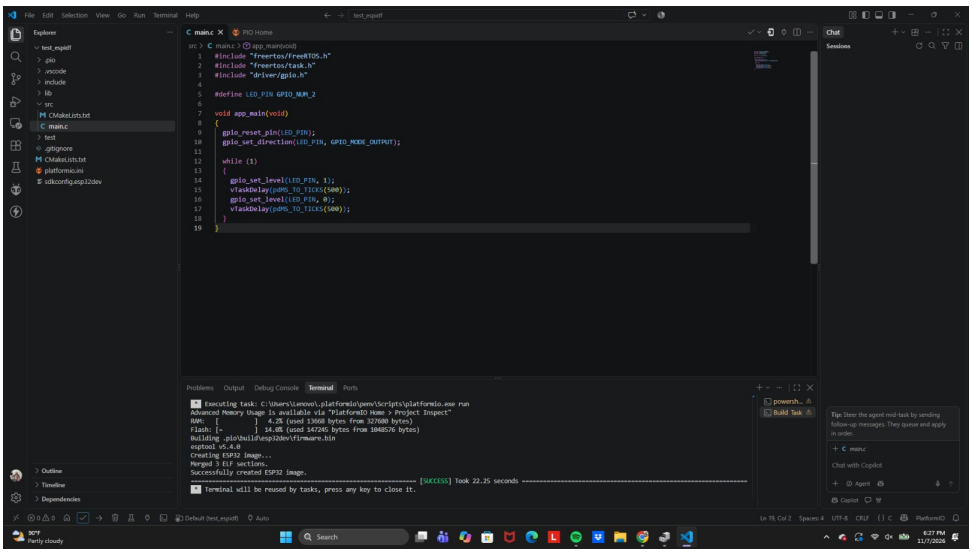
1.2 Percobaan 2: Blink ESP32 dengan ESP-IDF

1. Mengganti isi `platformio.ini` menjadi board `esp32dev` dengan framework `espidf`.



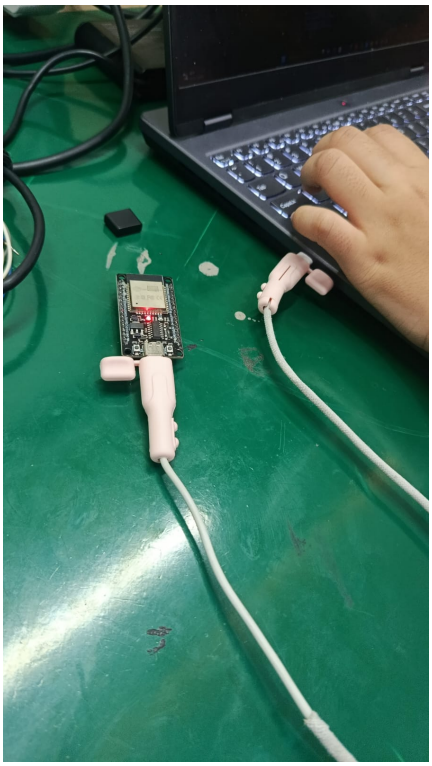
Gambar 5: Konfigurasi `platformio.ini` untuk ESP-IDF

2. Menghapus `src/main.cpp`, lalu membuat file `src/main.c` dan menulis kode Blink yang menggunakan fungsi `app_main()`.

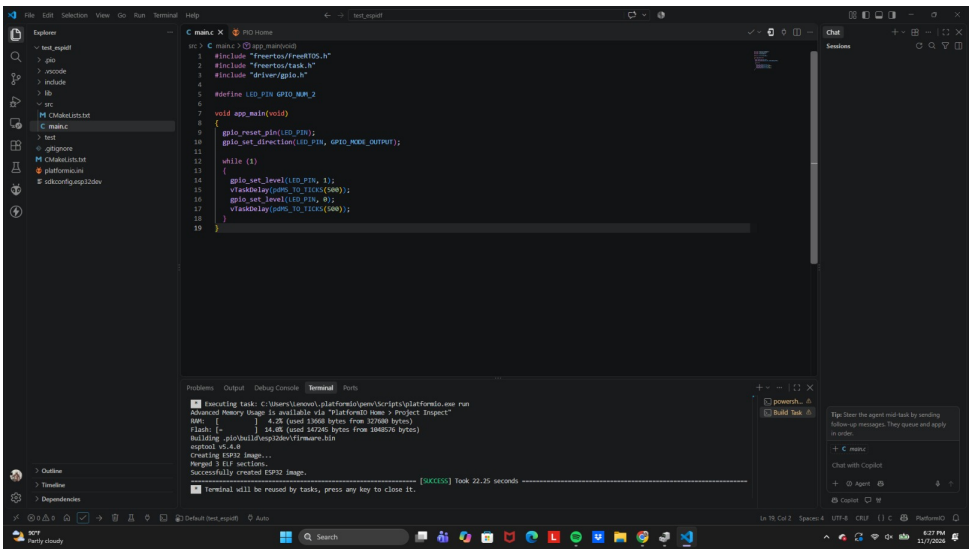


Gambar 6: Kode program Blink pada `main.c`

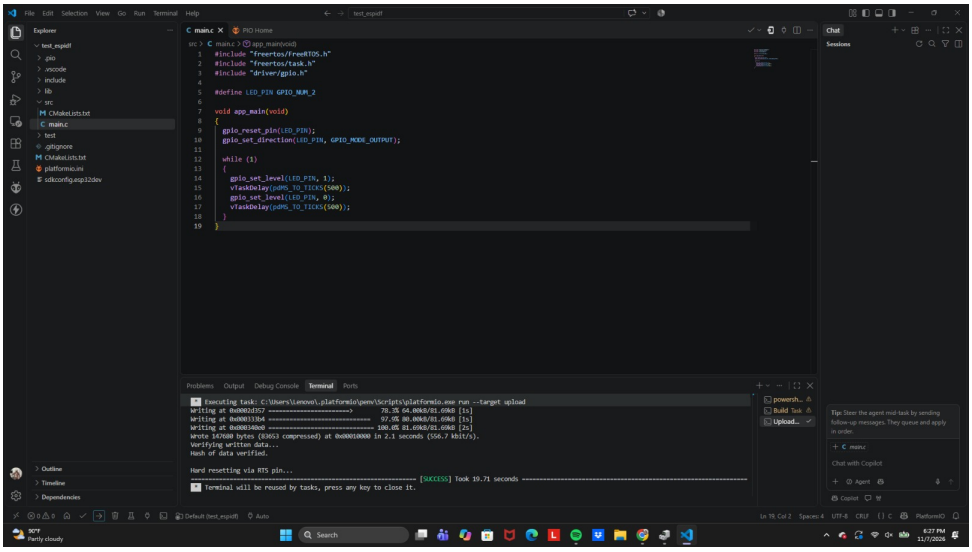
3. Menghubungkan ESP32 ke laptop menggunakan kabel USB, lalu melakukan Build dan Upload.



Gambar 7: ESP32 dihubungkan ke laptop melalui kabel USB

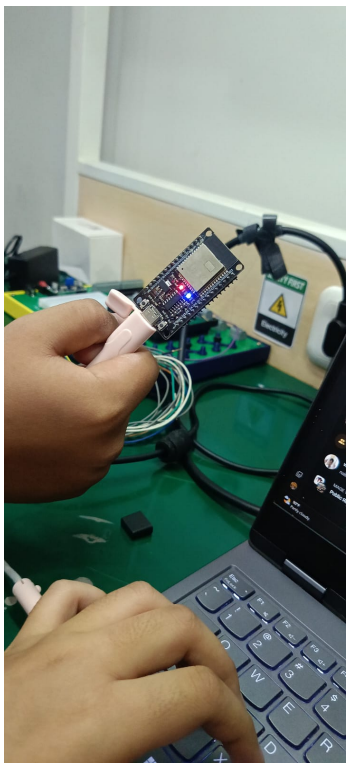


Gambar 8: Build program ESP-IDF berhasil



Gambar 9: Upload program ke ESP32 berhasil

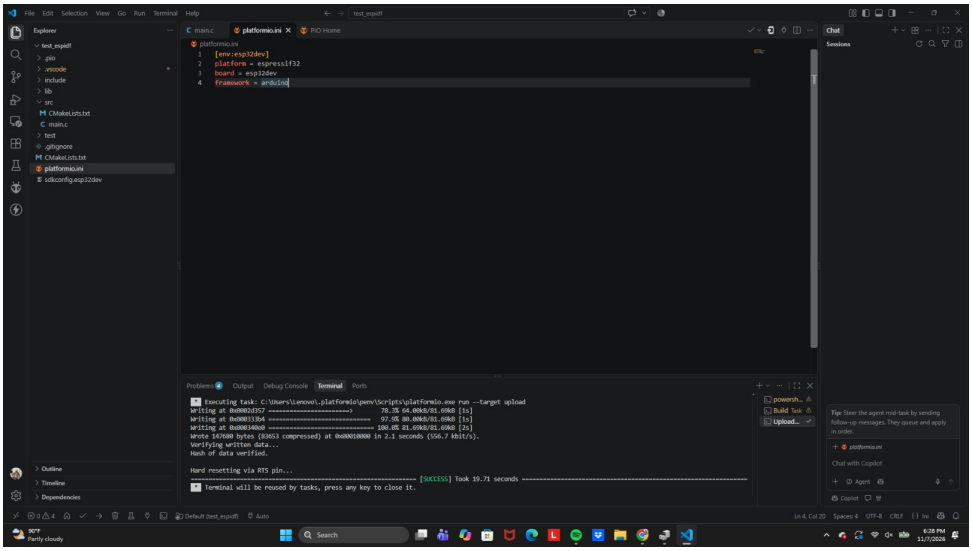
4. Mengamati LED onboard ESP32 pada GPIO 2.



Gambar 10: LED onboard ESP32 berkedip

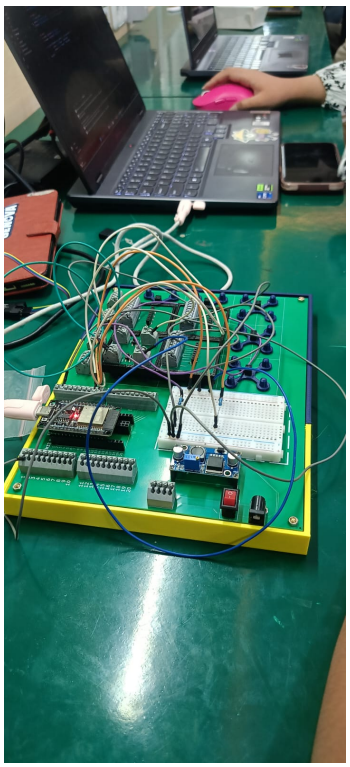
1.3 Percobaan 3: Pull-up dan Pull-down Eksternal vs Internal

1. Mengganti `platformio.ini` kembali ke framework Arduino, lalu mengubah `main.c` menjadi `main.cpp`.



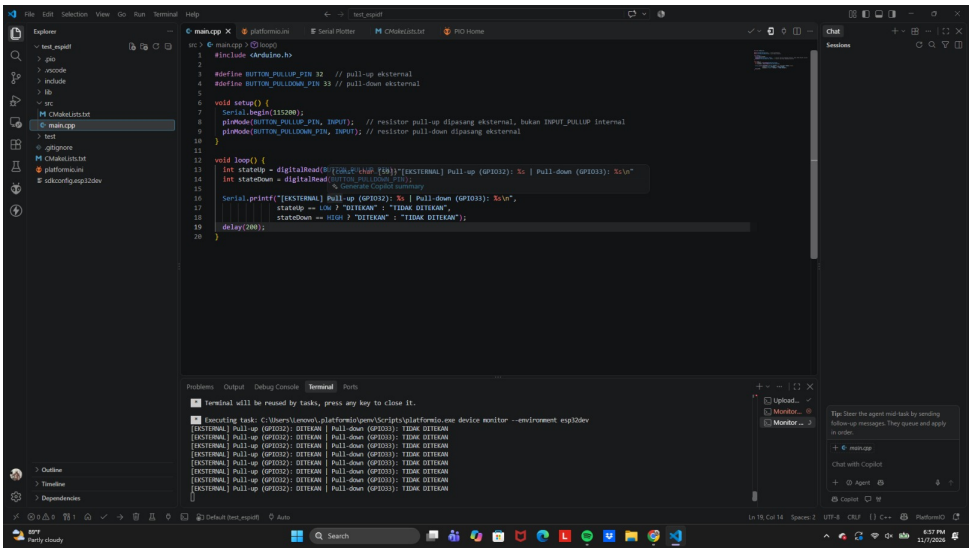
Gambar 11: Konfigurasi `platformio.ini` dengan framework Arduino

2. Merangkai dua pushbutton pada GPIO 32 (pull-up eksternal) dan GPIO 33 (pull-down eksternal), masing-masing dengan resistor 10 kΩ.



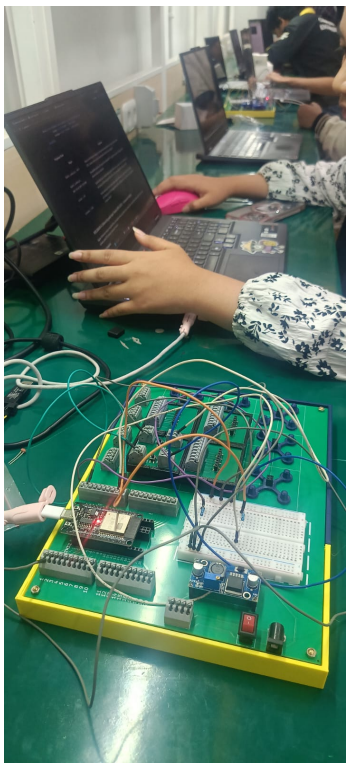
Gambar 12: Rangkaian pull-up dan pull-down eksternal

3. Mengupload Kode Program A (eksternal), lalu menekan tombol dan mengamati hasilnya pada Serial Monitor.



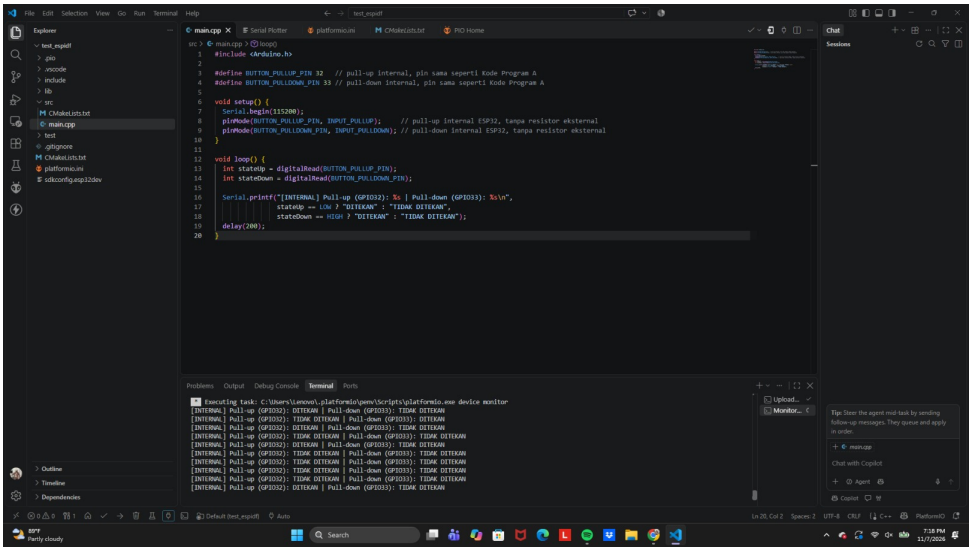
Gambar 13: Bagian A, tombol pull-up ditekan

4. Melepas sambungan resistor eksternal tanpa memindahkan tombol. Pada praktikum ini, resistor tetap berada di breadboard, tetapi kabelnya dipindahkan sehingga resistor tidak lagi terhubung ke pin GPIO.

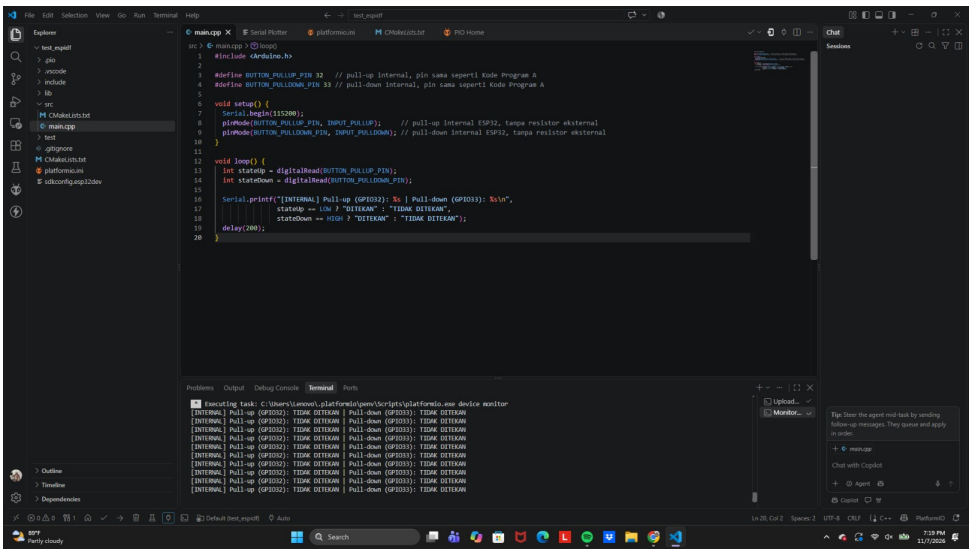


Gambar 14: Rangkaian tanpa resistor eksternal (kabel resistor dipindahkan)

5. Mengupload Kode Program B (internal) yang menggunakan `INPUT_PULLUP` dan `INPUT_PULLDOWN`, lalu menekan tombol yang sama.



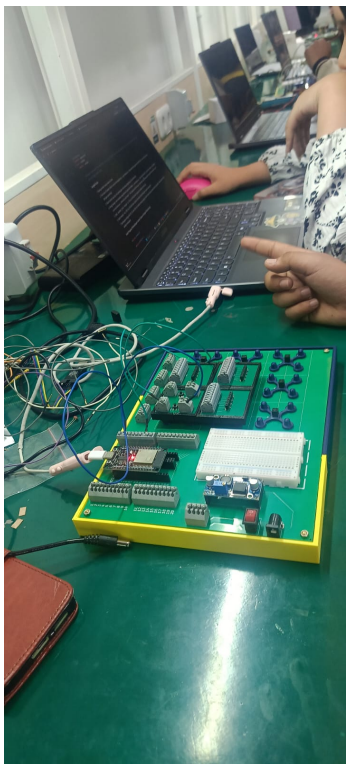
Gambar 15: Bagian B, tombol ditekan bergantian



Gambar 16: Bagian B, kedua tombol tidak ditekan

1.4 Percobaan 4: Debouncing

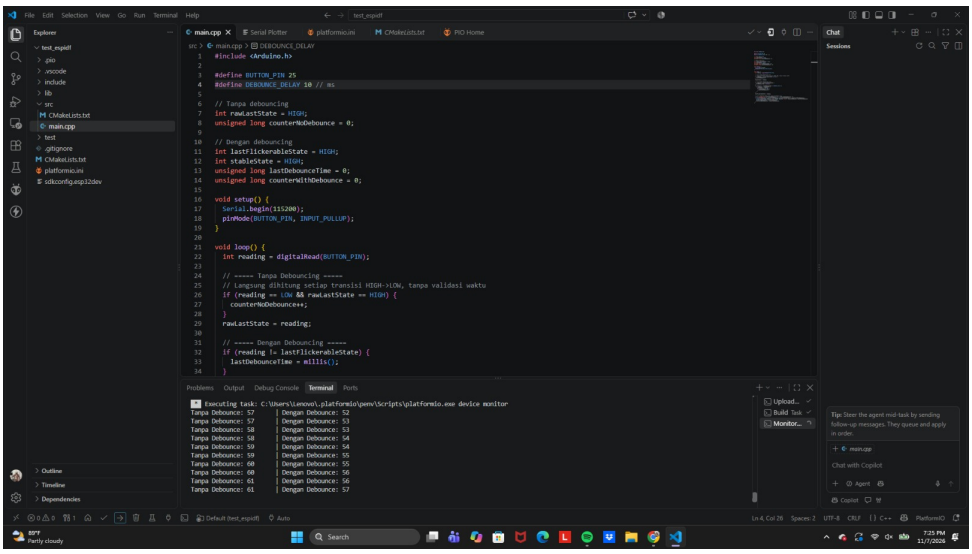
1. Merangkai satu pushbutton pada GPIO 25 dengan kaki lainnya ke GND, menggunakan pull-up internal.



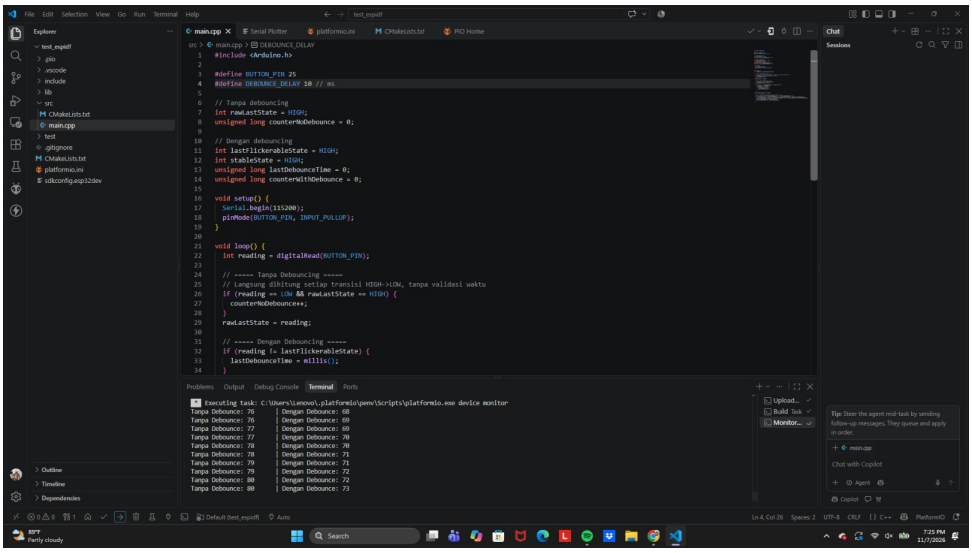
Gambar 17: Rangkaian Percobaan 4

2. Mengupload kode yang membandingkan `counterNoDebounce` dan `counterWithDebounce`.

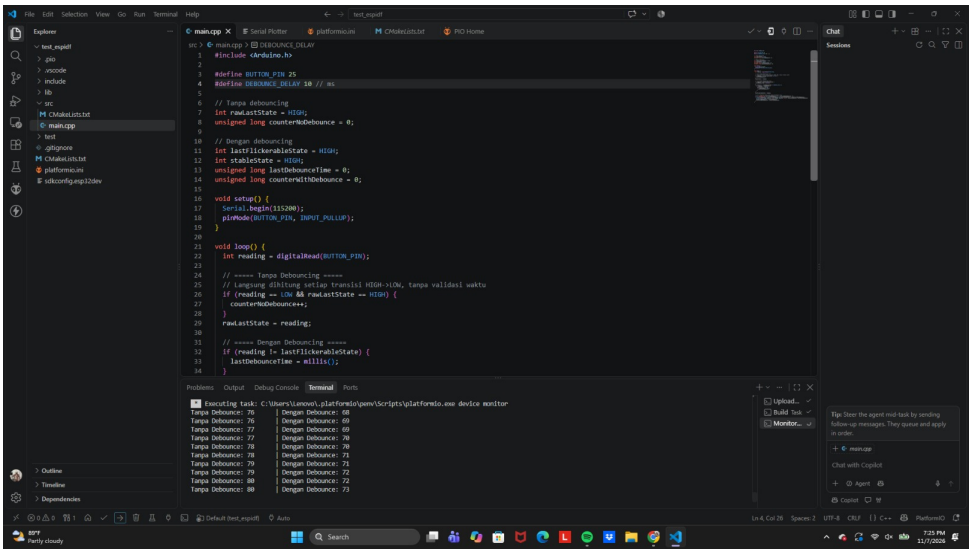
3. Menguji tombol dengan nilai `DEBOUNCE_DELAY` sebesar 10 ms, 50 ms, dan 200 ms. Pada setiap nilai, tombol ditekan secara lambat terlebih dahulu, kemudian secara cepat.



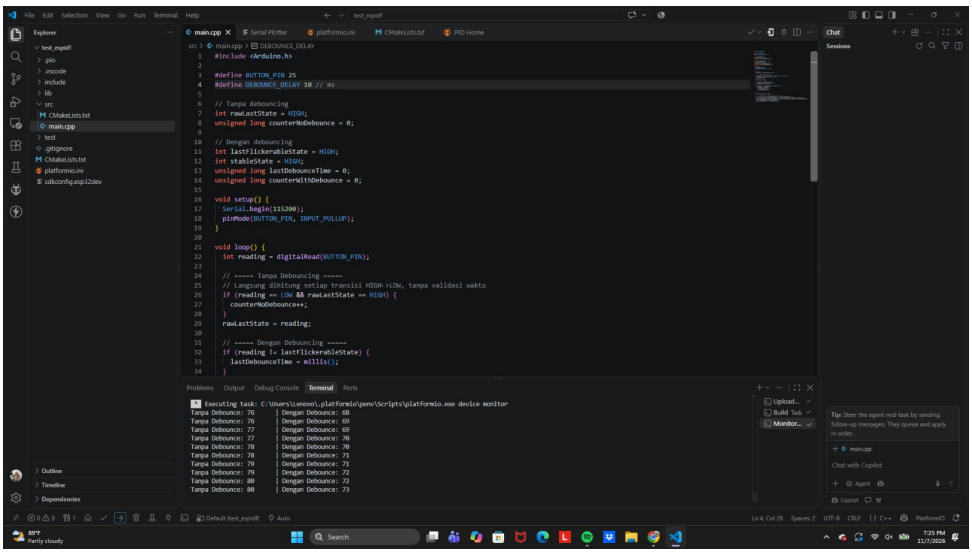
Gambar 18: Debounce 10 ms, tombol ditekan lambat



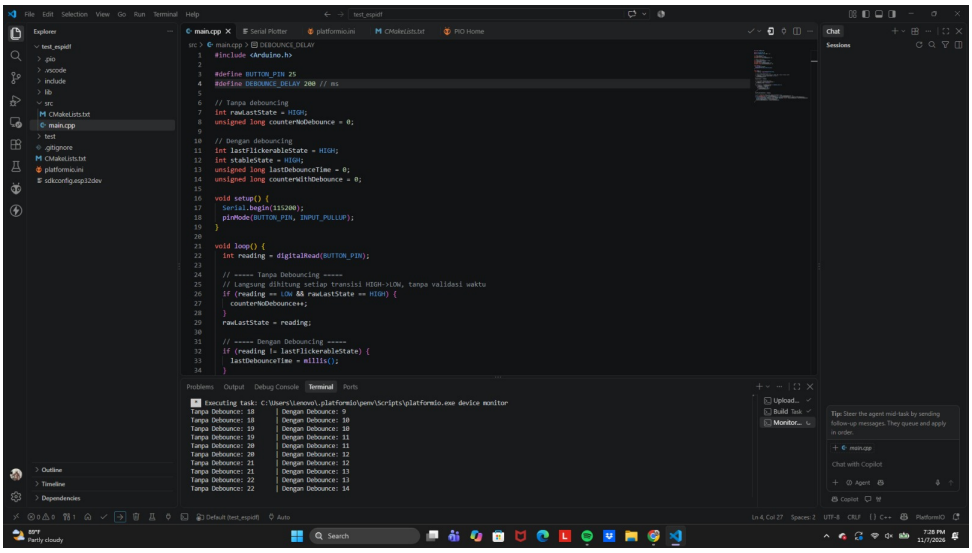
Gambar 19: Debounce 10 ms, tombol ditekan cepat



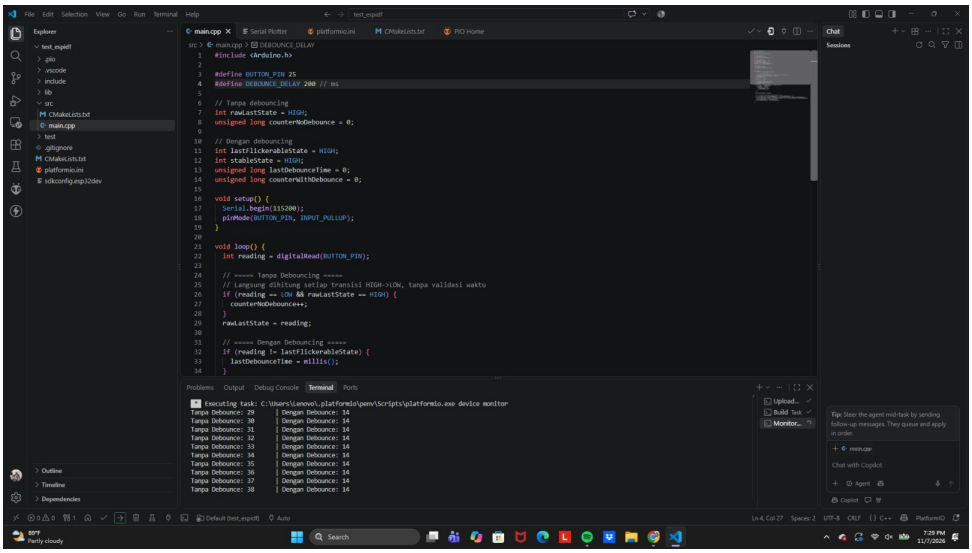
Gambar 20: Debounce 50 ms, tombol ditekan lambat



Gambar 21: Debounce 50 ms, tombol ditekan cepat



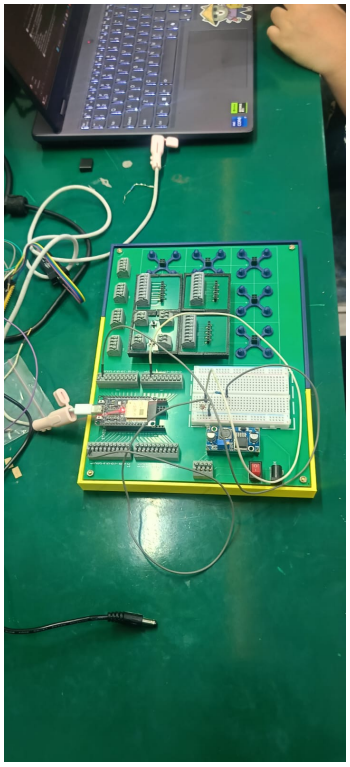
Gambar 22: Debounce 200 ms, tombol ditekan lambat



Gambar 23: Debounce 200 ms, tombol ditekan cepat

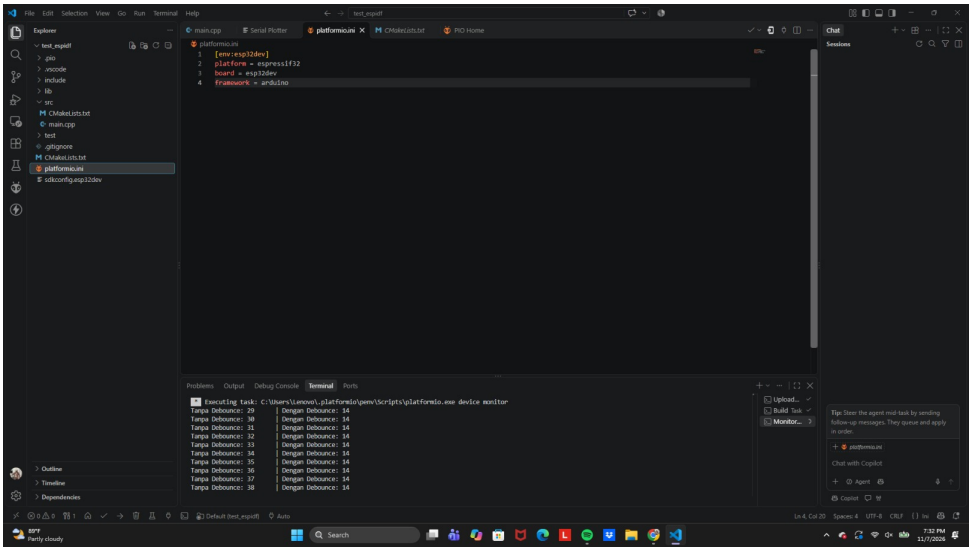
1.5 Percobaan 5: Pembacaan ADC dengan LDR

1. Merangkai LDR dan resistor 10 kΩ sebagai pembagi tegangan, dengan titik tengahnya dihubungkan ke GPIO 34.

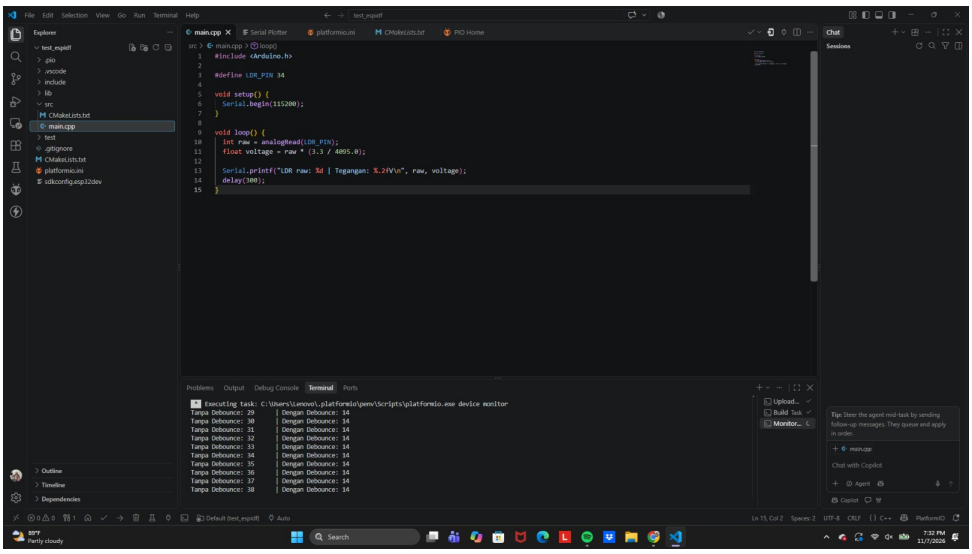


Gambar 24: Rangkaian Percobaan 5

2. Memastikan `platformio.ini` masih menggunakan framework Arduino, lalu mengupload kode pembacaan ADC dan membuka Serial Monitor.

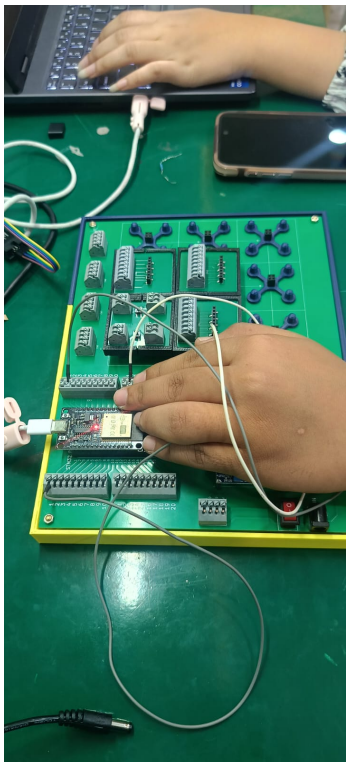


Gambar 25: Konfigurasi `platformio.ini` Percobaan 5

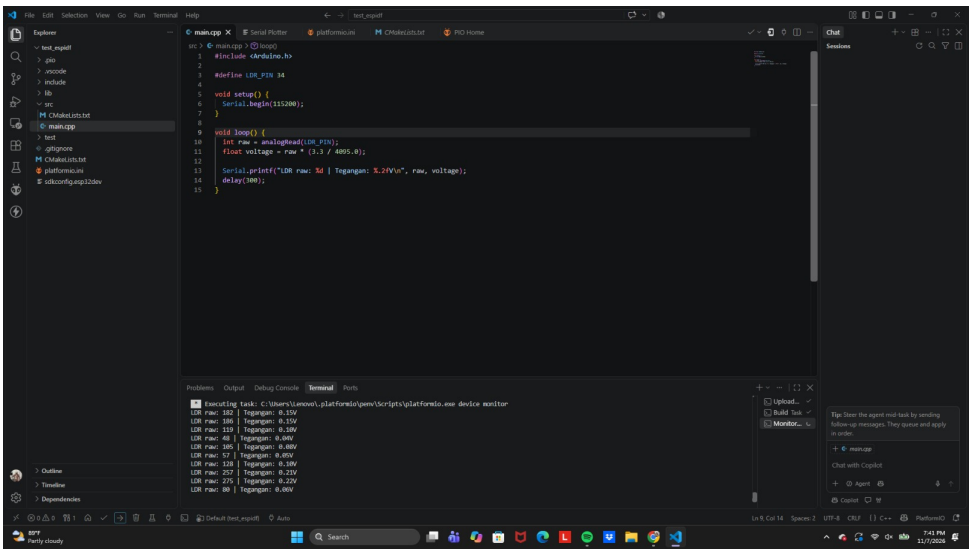


Gambar 26: Upload kode pembacaan ADC

3. Menutup LDR dengan tangan (kondisi gelap) dan mencatat nilai yang muncul.

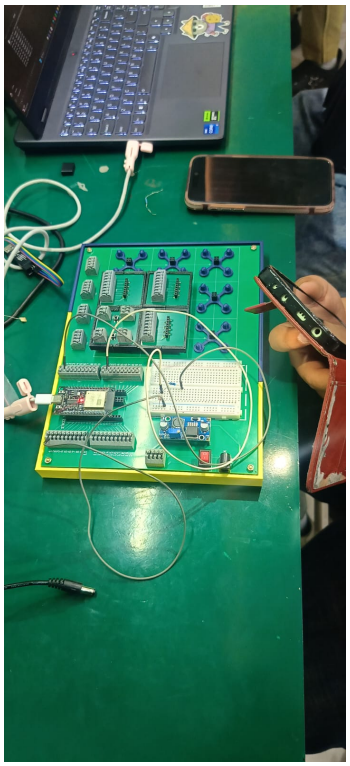


Gambar 27: LDR ditutup dengan tangan (kondisi gelap)

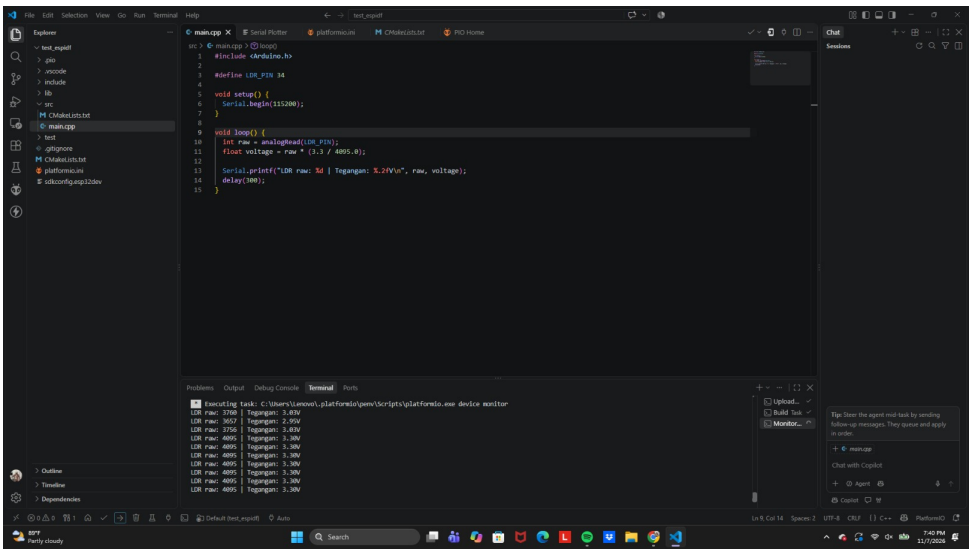


Gambar 28: Nilai ADC saat kondisi gelap

4. Menyinari LDR dengan lampu (kondisi terang) dan mencatat nilai yang muncul.



Gambar 29: LDR disinari lampu (kondisi terang)



Gambar 30: Nilai ADC saat kondisi terang

2 Analisis Hasil Percobaan

2.1 Percobaan 1: Blink STM32 Blackpill

Program Blink berhasil diupload ke STM32 Blackpill melalui ST-Link. Hal ini terlihat dari status SUCCESS pada terminal (Gambar 3), dan LED PC13 berkedip sesuai program (Gambar 4). Pada kode, LED diberi logika LOW agar menyala dan logika HIGH agar mati. Hal ini sesuai dengan teori bahwa LED onboard Blackpill bersifat aktif LOW.

Percobaan ini juga menunjukkan bahwa Blackpill tidak dapat diprogram hanya dengan kabel USB seperti ESP32. Blackpill tidak memiliki chip USB-to-Serial, sehingga program harus dikirim melalui ST-Link menggunakan jalur SWD. Oleh karena itu, keempat kabel ST-Link harus terpasang dengan benar. Jika SWDIO dan SWCLK tertukar, ST-Link tidak akan dapat menemukan chip STM32.

2.2 Percobaan 2: Blink ESP32 dengan ESP-IDF

Pada percobaan ini, ESP32 dapat langsung diprogram melalui kabel USB karena board-nya sudah memiliki chip USB-to-Serial. Build berhasil dalam waktu sekitar 22 detik (Gambar 8), upload juga berhasil (Gambar 9), dan LED pada GPIO 2 berkedip sesuai program (Gambar 10).

Perbedaan yang paling terasa dibandingkan Arduino terletak pada struktur programnya. ESP-IDF tidak menggunakan `setup()` dan `loop()`, melainkan fungsi `app_main()` yang berisi perulangan `while(1)`. Fungsi delay yang digunakan juga berbeda, yaitu `vTaskDelay()` dari FreeRTOS. Selain itu, kode ESP-IDF harus ditulis dalam file `.c`. Saat kode masih ditulis di `main.cpp`, build sempat gagal dengan pesan *undefined reference to app_main*. Masalah ini selesai setelah nama file diubah menjadi `main.c`. Proses build pertama ESP-IDF juga terasa lebih lama dibandingkan Arduino, karena seluruh komponen ESP-IDF perlu dikompilasi terlebih dahulu.

2.3 Percobaan 3: Pull-up dan Pull-down

Pada konfigurasi eksternal, saat tombol pull-up ditekan, Serial Monitor menampilkan Pull-up (GPIO32) DITEKAN, sedangkan Pull-down (GPIO33) tetap TIDAK DITEKAN (Gambar 13). Pin pull-up terbaca

LOW saat ditekan karena pin terhubung langsung ke GND. Sementara itu, pin pull-down tetap terbaca LOW karena tombolnya tidak ditekan dan pin ditarik ke GND oleh resistor. Hasil ini sesuai dengan teori pull-up dan pull-down.

Pada konfigurasi internal, saat kedua tombol tidak ditekan, Serial Monitor menampilkan TIDAK DITEKAN untuk kedua pin (Gambar 16). Artinya, GPIO 32 terbaca HIGH karena pull-up internal, dan GPIO 33 terbaca LOW karena pull-down internal. Saat tombol ditekan bergantian, status masing-masing pin berubah menjadi DITEKAN (Gambar 15). Hasil ini sama dengan konfigurasi eksternal, sehingga dapat disimpulkan bahwa resistor internal ESP32 dapat menggantikan resistor eksternal.

Pada praktikum ini, resistor eksternal tidak dicabut dari breadboard, melainkan kabelnya yang dipindahkan (Gambar 14). Cara ini tetap valid selama resistor benar-benar tidak terhubung lagi ke pin GPIO. Jika masih ada resistor yang terhubung, hasil pembacaan pada Bagian B bisa tercampur dengan pengaruh resistor eksternal. Kelebihan resistor internal adalah rangkaian menjadi lebih sederhana. Namun, fitur ini hanya tersedia pada pin tertentu. Pin GPIO 34 sampai 39 tidak memiliki resistor internal.

2.4 Percobaan 4: Debouncing

Pada percobaan ini, nilai `counterNoDebounce` selalu lebih besar daripada `counterWithDebounce`. Hal ini menunjukkan bahwa sebagian penekanan tombol mengalami bouncing, sehingga satu kali tekan terhitung lebih dari satu kali jika tidak disaring. Hasil pengujian untuk setiap nilai `DEBOUNCE_DELAY` adalah sebagai berikut.

- **10 ms.** Saat tombol ditekan lambat, nilai akhir tanpa debounce adalah 61 dan dengan debounce 57 (Gambar 18). Saat tombol ditekan cepat, nilainya menjadi 80 dan 73 (Gambar 19). Selisih kedua counter bertambah dari 4 menjadi 7. Artinya, jeda 10 ms masih terlalu pendek sehingga sebagian bouncing tetap lolos, terutama saat tombol ditekan cepat. Meskipun begitu, tombol terasa paling responsif.
- **50 ms.** Nilai 50 ms berada di antara kedua nilai lainnya. Jeda ini sudah cukup panjang untuk menyaring sebagian besar bouncing, sehingga satu kali penekanan tombol umumnya hanya terhitung satu kali oleh counter dengan debounce. Di sisi lain, jeda 50 ms masih cukup singkat, sehingga penekanan tombol yang cepat tetap dapat terbaca dan counter tidak tertahan seperti pada nilai 200 ms. Karena itu, nilai 50 ms memberikan keseimbangan antara pembacaan yang bersih dan respons tombol yang cepat.
- **200 ms.** Saat tombol ditekan lambat, tanpa debounce bernilai 22 sedangkan dengan debounce 14 (Gambar 22). Saat tombol ditekan cepat berulang kali, counter tanpa debounce terus naik dari 29 sampai 38, sedangkan counter dengan debounce tetap di angka 14 (Gambar 23). Hal ini terjadi karena jarak antar penekanan lebih singkat dari 200 ms, sehingga sinyal tombol tidak pernah dianggap stabil dan penekanan cepat tidak dihitung sama sekali.

Dari hasil tersebut, terlihat bahwa nilai `DEBOUNCE_DELAY` perlu dipilih dengan seimbang. Nilai yang terlalu kecil membuat tombol responsif, tetapi bouncing masih dapat lolos. Sebaliknya, nilai yang terlalu besar membuat hasil pembacaan lebih bersih, tetapi tombol menjadi lambat merespons dan penekanan cepat dapat terlewat. Oleh karena itu, nilai sekitar 50 ms umumnya digunakan sebagai titik tengah. Perlu diperhatikan juga bahwa bouncing tidak selalu muncul pada setiap penekanan, karena dipengaruhi oleh cara dan kekuatan tombol ditekan.

2.5 Percobaan 5: ADC dengan LDR

Saat LDR ditutup dengan tangan, nilai ADC berada pada kisaran 48 sampai 275, dengan tegangan sekitar 0,04 V sampai 0,22 V (Gambar 28). Saat LDR disinari lampu, nilai ADC naik menjadi 3657 sampai 4095, dengan tegangan sekitar 2,95 V sampai 3,30 V (Gambar 30). Perubahan ini terjadi karena hambatan LDR besar saat gelap, sehingga tegangan di titik tengah pembagi tegangan menjadi kecil. Saat terang, hambatan LDR turun dan tegangan pada GPIO 34 ikut naik.

Pada kondisi terang, nilai ADC sempat mencapai 4095. Angka ini merupakan nilai maksimum ADC 12 bit, yang berarti tegangan masukan sudah mencapai batas atas sekitar 3,3 V. Pada kondisi ini, cahaya yang lebih terang lagi tidak akan menghasilkan angka yang lebih besar.

Berbeda dengan input digital yang hanya mengenal HIGH dan LOW, ADC 12 bit pada ESP32 dapat menghasilkan 4096 nilai, yaitu 0 sampai 4095. Karena itu, perubahan intensitas cahaya dapat terbaca secara bertahap. Nilai yang sedikit naik turun pada kondisi yang sama disebabkan oleh noise dan perubahan cahaya kecil di sekitar sensor, misalnya posisi tangan yang tidak menutup LDR dengan sempurna. Nilai tegangan dihitung dengan rumus $V = \mathrm{raw} \times 3{,}3/4095$.

3 Hasil Tugas Modul

3.1 Rancangan Rangkaian

Simulasi dibuat menggunakan Wokwi dengan ESP32 DevKit. Pada matrix pushbutton, tiga pin baris digunakan sebagai output, sedangkan tiga pin kolom digunakan sebagai input dengan pull-up internal. Tombol dalam satu baris disambungkan satu sama lain, begitu juga tombol dalam satu kolom, sehingga sembilan tombol cukup dibaca menggunakan enam pin. Matrix LED disusun dengan cara yang sama, yaitu tiga pin baris terhubung ke anoda dan tiga pin kolom terhubung ke katoda melalui resistor 220 Ω.

Bagian	Pin ESP32
Baris tombol (output)	GPIO 27, 14, 13
Kolom tombol (input pull-up)	GPIO 33, 25, 26
Baris LED (anoda)	GPIO 23, 22, 21
Kolom LED (katoda)	GPIO 19, 18, 4

Tabel 1: Pembagian pin ESP32 pada tugas modul

Cara kerja programnya adalah sebagai berikut. ESP32 mengaktifkan baris tombol satu per satu dengan memberi logika LOW, lalu membaca ketiga pin kolom. Jika ada tombol yang ditekan pada baris tersebut, pin kolomnya akan terbaca LOW. Pembacaan ini kemudian disaring dengan debouncing 50 ms. Status tombol baru dianggap berubah jika sinyalnya sudah stabil selama 50 ms. Setiap tombol bekerja sebagai saklar *toggle*, yaitu sekali tekan menyalakan LED pada posisi yang sama dan tekan sekali lagi untuk mematikannya. Untuk menampilkan LED, ESP32 juga menyalakan baris LED secara bergantian dengan sangat cepat, sehingga semua LED terlihat menyala bersamaan.

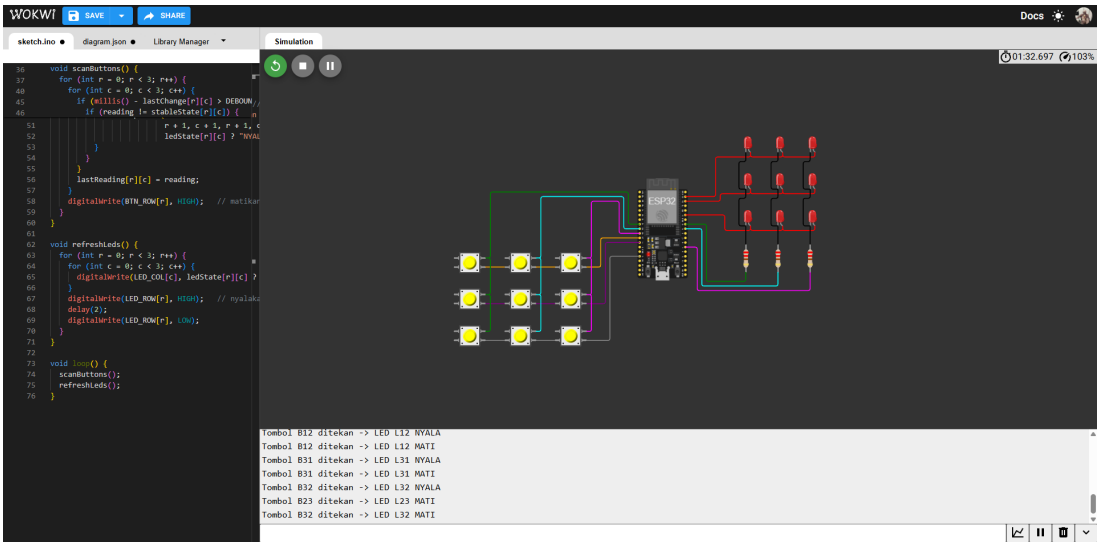
3.2 Source Code

```
1 #include <Arduino.h>
2
```

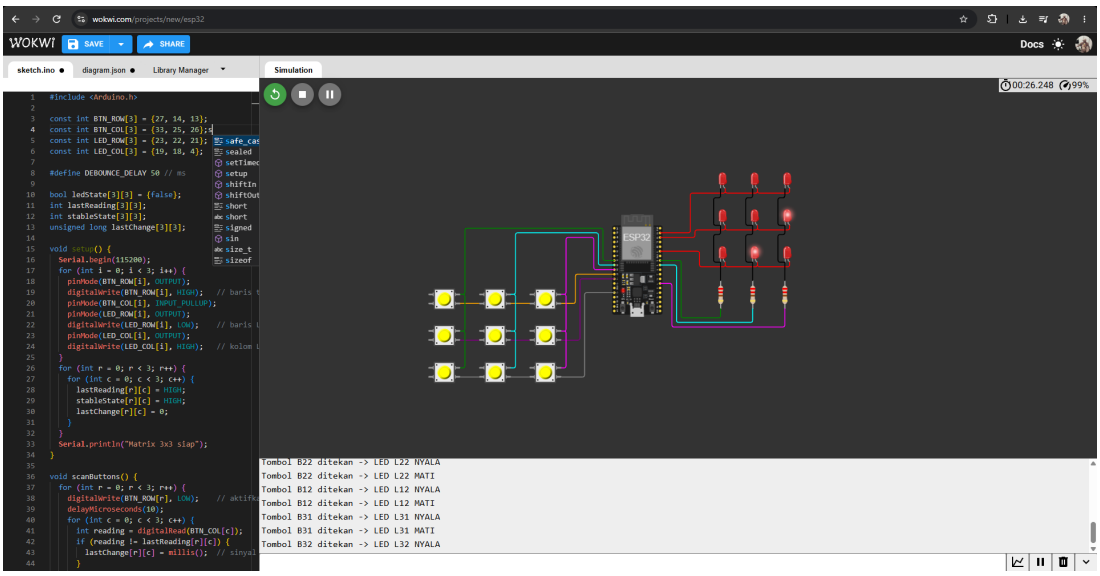
```
const int BTN_ROW[3] = {27, 14, 13};
const int BTN_COL[3] = {33, 25, 26};
const int LED_ROW[3] = {23, 22, 21};
const int LED_COL[3] = {19, 18, 4};

#define DEBOUNCE_DELAY 50

bool ledState[3][3] = {false};
int lastReading[3][3];
int stableState[3][3];
unsigned long lastChange[3][3];

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 3; i++) {
    pinMode(BTN_ROW[i], OUTPUT);
    digitalWrite(BTN_ROW[i], HIGH);
    pinMode(BTN_COL[i], INPUT_PULLUP);
    pinMode(LED_ROW[i], OUTPUT);
    digitalWrite(LED_ROW[i], LOW);
    pinMode(LED_COL[i], OUTPUT);
    digitalWrite(LED_COL[i], HIGH);
  }
  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      lastReading[r][c] = HIGH;
      stableState[r][c] = HIGH;
      lastChange[r][c] = 0;
    }
  }
  Serial.println("Matrix 3x3 siap");
}

void scanButtons() {
  for (int r = 0; r < 3; r++) {
    digitalWrite(BTN_ROW[r], LOW);
    delayMicroseconds(10);
    for (int c = 0; c < 3; c++) {
      int reading = digitalRead(BTN_COL[c]);
      if (reading != lastReading[r][c]) {
        lastChange[r][c] = millis();
      }
      if (millis() - lastChange[r][c] > DEBOUNCE_DELAY) {
        if (reading != stableState[r][c]) {
          stableState[r][c] = reading;
          if (stableState[r][c] == LOW) {
            ledState[r][c] = !ledState[r][c];
            Serial.printf("Tombol B%d%d ditekan -> LED L%d%d %s\n",
                          r + 1, c + 1, r + 1, c + 1,
                          ledState[r][c] ? "NYALA" : "MATI");
          }
        }
      }
      lastReading[r][c] = reading;
```

```
    }
    digitalWrite(BTN_ROW[r], HIGH);
  }
}

void refreshLeds() {
  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      digitalWrite(LED_COL[c], ledState[r][c] ? LOW : HIGH);
    }
    digitalWrite(LED_ROW[r], HIGH);
    delay(2);
    digitalWrite(LED_ROW[r], LOW);
  }
}

void loop() {
  scanButtons();
  refreshLeds();
}
```

3.3 Hasil Simulasi



Gambar 31: Simulasi saat semua LED dalam keadaan mati



Gambar 32: LED menyala sesuai posisi tombol yang ditekan

Gambar 31 menunjukkan rangkaian saat semua LED dalam keadaan mati. Setelah beberapa tombol ditekan, LED pada posisi yang sama menyala, seperti terlihat pada Gambar 32. Serial Monitor juga menampilkan tombol mana yang ditekan beserta status LED-nya. Setiap tombol hanya mengubah status LED satu kali untuk satu penekanan. Hal ini menunjukkan bahwa debouncing pada program sudah berjalan dengan baik.

3.4 Jawaban Pertanyaan Analisis

1. Debouncing tetap diperlukan karena bouncing tidak ditentukan oleh banyaknya tombol yang ditekan, tetapi berasal dari tombol itu sendiri. Seperti yang dijelaskan pada modul, kontak logam di dalam tombol sempat memantul beberapa kali sebelum benar-benar menempel. Jadi, walaupun hanya satu tombol yang ditekan, sinyal yang terbaca tetap bisa berubah-ubah antara HIGH dan LOW selama beberapa milidetik. Pada rancangan ini, ESP32 membaca seluruh tombol secara berulang dengan sangat cepat, sehingga pantulan tersebut sangat mungkin terbaca sebagai beberapa kali penekanan. Hal ini juga sudah terbukti pada Percobaan 4, ketika counter tanpa debounce selalu lebih besar daripada counter dengan debounce. Karena LED pada rancangan ini bekerja secara toggle, tanpa debouncing LED bisa menyala lalu langsung mati lagi padahal tombol hanya ditekan satu kali. Oleh karena itu, debouncing 50 ms tetap diterapkan pada setiap tombol.

2. Rancangan ini menggunakan pull-up internal (`INPUT_PULLUP`) pada pin kolom tombol. Pull-up dipilih karena baris tombol diaktifkan dengan logika LOW, sehingga tombol yang ditekan akan menarik pin kolom menjadi LOW. Saat tombol tidak ditekan, pin kolom terbaca HIGH karena ditarik ke 3,3 V oleh resistor internal ESP32. Pin kolom tidak boleh dibiarkan floating, karena pin yang tidak terhubung ke mana pun nilainya bisa berubah-ubah secara acak akibat noise. Jika hal ini terjadi, ESP32 bisa mengira ada tombol yang ditekan padahal tidak, dan LED pada matrix bisa menyala dengan sendirinya.

3. Perbedaan mendasarnya terletak pada jumlah kondisi yang bisa dibaca. Saat GPIO dibaca sebagai input digital, ESP32 hanya membedakan dua kondisi, yaitu HIGH dan LOW. Cara ini

sudah cukup untuk tombol, karena tombol memang hanya memiliki dua keadaan, yaitu ditekan atau tidak. Jika salah satu LED diganti dengan LDR dan dibaca melalui ADC, yang dibaca adalah besar tegangannya, lalu tegangan tersebut diubah menjadi angka. ADC 12 bit pada ESP32 membagi rentang 0 sampai 3,3 V menjadi 4096 tingkat, sehingga perubahan tegangan yang kecil pun tetap bisa terbaca. Hal ini sesuai dengan hasil Percobaan 5, ketika nilai LDR berada di sekitar 48 sampai 275 saat gelap dan naik hingga 3657 sampai 4095 saat terang. Karena itu, ADC dapat menunjukkan tingkat cahaya secara bertahap, tidak hanya terang atau gelap seperti pada input digital.

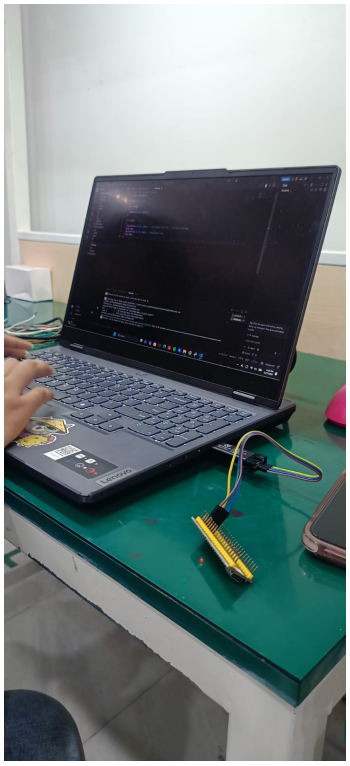
4 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. STM32 Blackpill memerlukan ST-Link untuk proses flashing melalui jalur SWD karena tidak memiliki chip USB-to-Serial, sedangkan ESP32 dapat diprogram langsung melalui kabel USB.
2. Framework Arduino lebih sederhana karena menggunakan struktur `setup()` dan `loop()`. Sementara itu, ESP-IDF menggunakan `app_main()` berbasis FreeRTOS dan kodenya harus ditulis dalam file `.c`.
3. Pada konfigurasi pull-up, pin terbaca HIGH saat tombol tidak ditekan dan LOW saat ditekan, sedangkan pull-down berlaku sebaliknya. Resistor internal ESP32 memberikan hasil yang sama dengan resistor eksternal.
4. Debouncing berhasil mengurangi pembacaan ganda akibat bouncing. Nilai jeda 10 ms masih meloloskan sebagian bouncing, sedangkan nilai 200 ms membuat penekanan cepat tidak terbaca sama sekali.
5. ADC 12 bit pada ESP32 dapat membaca perubahan cahaya pada LDR secara bertahap, dari sekitar 0,04 V saat gelap hingga 3,30 V saat terang.
6. Pada tugas modul, matrix pushbutton 3x3 berhasil mengendalikan matrix LED 3x3 menggunakan 12 pin GPIO ESP32, dengan debouncing 50 ms pada setiap tombol.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 33: Dokumentasi saat mengerjakan praktikum